

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/342416461>

An Ontology of IS Design Science Research Artefacts

Chapter *in* Lecture Notes in Business Information Processing · June 2020

DOI: 10.1007/978-3-030-50316-1_8

CITATIONS

3

READS

131

3 authors, including:



Hans Weigand
Tilburg University

222 PUBLICATIONS 3,310 CITATIONS

[SEE PROFILE](#)



Paul Johannesson
Stockholm University

285 PUBLICATIONS 4,520 CITATIONS

[SEE PROFILE](#)

An Ontology of IS Design Science Research Artefacts

Hans Weigand^[0000-0002-6035-9045], Paul Johannesson^[0000-0002-7416-8725] and Birger Andersson[□]

¹ Tilburg University, P.O.Box 90153, 5000 LE Tilburg, The Netherlands
{H.Weigand}@uvt.nl

² Stockholm University, Department of Computer and Systems Sciences, Sweden
{pajo,ba}@dsv.su.se

Abstract. From a design science perspective, information systems and their components are viewed as artefacts. However, not much has been written yet on the ontological status of artefacts or their structure. After March & Smith's (1995) initial classification of artefacts in terms of models, constructs, procedures and instantiations, there have been only a few attempts to come up with a more systematic approach. After reviewing previous work, this conceptual paper introduces an ontology of IS artefacts. It starts with an ontological characterization of artefacts and technical objects in general and proceeds to introduce a systematic classification of IS artefacts and compare it with existing work. We end with some practical implications for design research.

Keywords: Design Science, IT artefact, ontology

1 Introduction

Design Science Research [16] has grown in popularity in the Information Systems field. Whereas empirical sciences acquire knowledge about the natural world (physics) or human behavior (social sciences), Design Science Research (DSR) is interested in IT artefacts, such as algorithms, methods and modeling languages, and the effectiveness of their use. Because of its focus on artificial objects, DSR has similarities with traditional engineering sciences and medicine. Building artefacts and evaluating artefacts in context is often seen as the core of DSR. But what is an artefact. Is artefact the same as “artificial object” [30]? Can we call any solution an artefact? What exactly do we say when we call something an IT artefact? Although by now there is quite a substantial literature about design science, in the form of research articles and textbooks, a systematic investigation of the design artefact concept is still missing. Building on our general ontology of artefacts (in progress), the goal of this conceptual paper is to introduce an ontology of IS artefacts and analyze the practical implications for Design Science Research.

The structure of this paper is as follows. In Section 2, we recapitulate what has been written on the artefact concept in the (broad) DSR literature. Section 3 is a short overview of our general artefact ontology. On this basis of this ontology, we analyze the

main IS artefacts and their interrelationships in Section 4, compare the results with the existing literature and discusses practical implications for DSR researchers.

2 Background

A general understanding of artefact in Design Science is that is an object made by humans with the intention that it is used to address a practical problem [3,18]. Design Science arose in the 1990's within the field of Information Systems and although its idea of artefacts included physical artefacts like hammers or medical drugs, its interest went mainly in the direction of symbolic artefacts such as encountered in the Information System domain. Simon's *The Sciences of the Artificial* [30] focused on computers and artificial intelligence software viewed as artificial objects. Although Simon is not very explicit, he seems to regard a design object as more specific than an artificial object, witness the fact that he views markets being the result of a social evolution process as artificial objects but without a designer. An also very broad but different definition is expressed by Gregor and Hevner [14:4]: 'We argue toward a more expansive view of the IT artifact to include any designed solution that solves a problem in context'. Wieringa's [37:29] definition is similar ('an artefact is something created by people for some practical purpose'). He prefers to talk about treatments. What the design researcher designs is a treatment, that is, 'a desired interaction between the artefact and the problem context' and he defines design as 'a decision about what to do'. There is a risk that the artefact concept loses its meaning if it includes any decision or any solution. To keep the connection with both common sense and the technical sciences and to be able to give substance to the design aspect, an artefact must have something of "an object made by humans". At the same time, we think a proper definition should do justice to the intuitions of Gregor and Wieringa and this is one of the challenges for an ontology of artefacts.

2.1 Design research outcome: artefacts vs theory

According to [28], characteristic for design research is the situated implementation of an invention (artefact) as software or system. Apart from that, he argues that design research should produce knowledge and understanding, which sometimes he also calls artefacts. Pursuing this second suggestion, [38] proposes "theory" to be a relevant artefact type. What we see as problematic here is that the artefact is identified with (design) research outcome. Although the authors are right in stressing that design research should not just produce an instantiated artefact but contribute to theory [13], it is confusing to take this scientific theory to be the design artefact. That would make all sciences design sciences.

Goldkuhl & Lind [11] make a distinction between artefacts as objects of empirical design practices and abstract design knowledge (see also more recently [9]). This is a useful distinction to start with, as long as we do not conflate again the "meta-artefacts" (their terminology) on the abstract design level with design knowledge *about* these artefacts. This knowledge requires an identification of the meta-artefact but goes beyond

the identification by developing, for instance, mathematical models of the artefact or by accumulating experimental results about its performance.

As science in general abstracts from the concrete, so design knowledge abstracts from specific application contexts. This holds in particular for design knowledge as pursued in Design Science Research. However, the level of abstraction is usually not as high as e.g. in physics. Wieringa [37:10] uses the term “middle-range generalization”. In the opposition concrete vs. abstract it is often overlooked that there is also an *aggregation* dimension [31]. For instance, in the IT field, there are infinitely many possible programs, with all kinds of variations. These are not all interesting DSR artefacts. IT researchers are typically more focused on developing *components*, e.g. in the form of a method library, than end products. The components are not necessarily more *abstract* than integrated systems, but more *elementary*. It makes sense to see these components as tangible DSR outcomes, and the knowledge thereof as design knowledge. In contrast, evaluation studies in the domain of Information Management typically look at the end product (as blackbox) in some use context. The intended outcome of this type of research does not consist of new artefacts, but of knowledge on the use properties and the effectiveness of some artefact type (e.g. the type ERP system). In our view, the IT and IM perspective complement each other. Sometimes, it is the case that the IT research provides knowledge about the technical feasibility and performance of the artefact and the IM research extends the design knowledge by studying its effects in context. In other cases, the two work on different aggregation levels: the IT research studying the effects of some component in the use context of the integrated IT system, and the IM research looking at the effects of the integrated system in the use context of an enterprise.

2.2 Socio-technical systems

We consider IS artefacts to be research objects or units of analysis - what the design knowledge is about. From a Computer Science perspective, the Information System is the information technology that does data processing. The management view on Information Systems sees the object as much broader, including the business processes that use the IT, the humans involved and the organizational structure around it. The Information System, defined in this way, can be called a socio-technical system as it includes a technical component but also people. Sometimes the IT is called a socio-technical system. According to [29:82–83] IT artifacts ‘have both technical and social design features and are therefore better regarded as SocioTechnical (ST) artifacts’....‘We define the IT artifact as a sociotechnical assemblage.’ As pointed out, among others, by Alter [2], we should be cautious with terms like socio-technical artefacts or ensemble artefacts, as if there are also other artefacts that do not have social design features. Arguably, (cf. [36]) a software application, as any design artefacts, exist in the historical reality in which technical aspects and social aspects (in a broad sense) are intertwined in many ways. The application has been designed by humans, driven consciously or unconsciously by social values. Once implemented it falls under legal rules, has an economic value, has an effect on social relationships etc. However, that does not make these applications socio-technical in the original sense [34].

[2] provocatively suggests avoiding the vague term IT artefact completely, especially when it is applied just to something containing IT, like an airplane, or related to IT, like user training. In the following, we will keep using the terms IT and IS artefact. A precise characterization of the artefact concept and the exploration of all meaningful artefacts in the IS domain is the subject of this paper. We do not need a special category of socio-technical artefacts. A socio-technical system is a *system* that includes people. This is a meaningful concept, as there are also many systems that do not include people.

2.3 Artefact vs system

Both in Simon [30] and in more recent publications [22] the words artefact and system are sometimes used as synonyms. However, this is problematic. A system is defined as a composite of elements with attributes and relationships between them that makes a meaningful whole. ‘The idea of holism which assumes that particular parts can be described only by the knowledge of their meaning and position in the whole, is the basis of the conception of the perception of reality as complex systems’ [12]. From its early beginnings, system theory embraces both natural and artificial (man-made) systems. System theory is not so much about a particular class of objects, but is a particular view: the holistic view, in contrast to a reductionist view. The market can be seen as a complex system, or a family. So clearly not every system is an artefact or even an artificial object. One could say that an artefact is a system, as indeed the elements of a composite artefact have a function in the whole, so cannot be characterized exhaustively on their own. A system view on the artefact deemphasizes the role of the designer intention and focuses on understanding how the overall behavior derives or emerges from its parts. In our view, it makes sense to include somehow the system view to the Design Science artefact definition. However, if the Information System is conceived as including humans, then the system is not the output of intentional design. Many elements of the system, not only the IT but also e.g. an authorization scheme or a business process structure, can be designed, and may be viewed as artefacts (or part of the IT artefact).

That we must be cautious to equate system and artefact is supported, be it in other wordings, by management researchers. [27] expressed scepticism about the work of Van Aken [1] and others who advocate a design research approach to management. If design is taken in the traditional engineering sense, then it only has a limited applicability in management. They describe organizations as social artefacts that evolve over time in unpredictable ways. This evolution could be called *path-creation design*. In our wordings, what is meant here is that the evolution of the organization is not determined by physical laws mobilized by intentional design but includes human creative intervention. We agree. We support a worldview that acknowledges human creativity [36]. The question is whether the word (path-creation) design is appropriate and in what sense the organization is a social artefact (cf. [19] for a discussion of different interpretations of the term “design thinking”).

2.4 Typology of IS artefacts

Some work has been done on a typology of IS artefacts, but surprisingly little. [23,16] describe artefacts as the outcome of design research, distinguishing four typical types: constructs, models, methods and instantiations. The motivation is brief, but it seems that for March and Smith, *models* are at the core. Constructs provide the language to build models. Methods are described as ways to implement models, and instantiations as the integrated result. The March and Smith proposal has been broadly accepted. Alternatively, based on a literature review of actual design research projects, [26] identified eight relevant artefact types: notation, requirement, metric, system design, method, algorithm, guideline and pattern. The paper does not try to analyze ontologically how these types are related. It should also be noted that the scope of the literature review was mainly on design research conferences, and not the IS literature as a whole.

A different way of classifying IS artefacts is by means of their function. Iivari [17] distinguished seven functions: to automate, to augment, to mediate, to inform, to entertain, to create art, to accompany. Each function characterizes an archetypical IT application. In contrast to March & Smith's general design science approach, Iivari's classification is more specific to the IS domain, but it also lacks ontological analysis. It can be used to classify IT artefacts on the basis of their use.

2.5 Ontology of IT artefacts

[35] provided a detailed ontological analysis of *software*. They follow Osterweil in describing software as something non-physical and intangible. They do not agree with proposals that identify the program with the code. The reason given for the distinction is that software can be updated, so can have different codes at different times, while a code becomes different by each change (the paper talks about a change in the syntactic structure that includes variable renaming and inclusion of a comment, so apparently any change apart from layout). This latter claim is disputable. As in the famous Ship of Theseus example, it is possible to change a part (replace a plank) while the ship keeps its identity.

According to [35] 'when a code is in the circumstance that somebody, with a kind of *act of baptism*, intends to produce certain effects on a computer, a new entity emerges ... a computer program'. Again, we have some remarks. First, it feels odd that the code is presented to be there *before* the program emerges. Usually, there is a program design first. Secondly, the paper continues to write that the program is not a physical object as it lacks a location in space – but why is code inside space and the baptized code suddenly outside space? In our view, the issues are caused by a lack of attention to the design itself. We have a problem with calling a program instance non-physical and at the same time saying that it is constituted by (physical) code. However, we do agree that code and program must be distinguished, and also that a program can be distinguished from a software system (where the former has an effect in the computer internally and the latter externally, in the interface).

Coming from outside the IS domain, the Information Artefact Ontology (IAO, [7]) provides an ontological account of information content entities (ICE) such as document, databases and digital images. An ICE is defined as an entity that is ‘generically dependent on some material entity and which stands in a relation of aboutness to some [other] entity’. The material entity can be the paper, the other entity can be the person depicted in the digital image ICE. An ICE can be concretized in an information carrier (e.g. a copy of the image). In [8], a distinction is made between the information artefact (a material entity) and the information carrier, now called information quality entity. The information artefact is the disk or paper itself - viewed purely physically, that is used to *bear* information. Although our ontology will be structured differently, with more emphasis on the design, we concur with IAO with respect to the material basis of information artefacts. [8] is a detailed analysis of the “aboutness” relation, and how to deal with cases where the entity that the information content entity is about does not exist. In our view, this analysis is useful when we are talking about language and mental states. However, we regard that as belonging to language ontology, which is different from - although presupposed in - an information object/text ontology.

3 Ontology of artefacts

Although the focus of this paper is on *IS artefacts*, we start with a short summary of our general ontology of artefacts (in progress) which builds on and extends the work of Borgo [5] as well as Lawson [21] and Kroes [20]. Our aim is to integrate their work with the Design Science Research perspective, such as described in Section 2, and ground it in the foundational ontology UFO [15]. In this Section, we first summarize the artefact ontology and illustrate it with some small examples. In 3.4, we focus on text artefacts, and in 3.5 we define DSR artefacts.

3.1 Basic definitions

An **artificial object** is a physical object intentionally or unintentionally created or adapted by humans. A **natural object** is a physical object that is not an artificial object. For the definition of physical object we follow the UFO definition of a *substance individual*, that is ‘an endurant that consists of matter (i.e., is “tangible” or concrete), possesses spatio-temporal properties, and can exist by itself; that is, it does not existentially depend on other endurants, except possibly on some of its parts)’ [15]. The distinction artificial object/natural object does not necessarily exhaust the complete set of endurants. For instance, mathematical objects and social entities are endurants but not physical objects. UFO does not split up the world of substantials into artificial and natural, but into agents and objects; and distinguishes social objects from physical objects.

Instrument is a role of an object in an event or activity. The instrument is *used* in the event. This corresponds partially to the UFO definition of resource as ‘an object participating in an action’. However, some resources are consumed in the use (e.g. a medicine), and cease to exist, while others are not. We use the role instrument for the latter persisting participating objects only. Note that instruments are not necessarily

artificial objects: a shell can be used for drinking water, stars can be used to navigate on sea.

A **technical object** is an *artificial object* (individual) that is made by humans for some purpose (intentionally) whose capacities are (in principle) exhaustively described by the physical laws *and* the intentional design. A technical object has *attributed capacities*¹ that are *realized* in the use.

An **artefact** like “the Diesel engine” is a universal (1stOT in MLT [6] - with artefact a 2ndOT category) that has a design specification consisting of a (at least one) *make plan*, a (at least one) *use plan* and a *capacity specification*. An individual object is said to *conform* to the artefact if its composition is according to the make plan, it shows the specified capacities, and its use intention corresponds to the use plan (for convenience, we also talk about **artefact instances** for these conforming individuals). We talk about **technical artefacts** when the objects conforming to the artefact are *technical objects*, and non-technical artefacts when there are objects conforming to the artefact but not technical. The *make plan* basically expresses the composition of the object in terms of **components** and allows reproducing individuals conforming to the artefact. The *use plan* specifies at least in which events and under which use conditions the object can be used to achieve which intended effect. To delimit the scope of artefacts to those products that have “a practical purpose”, we require the use plan to refer to a *use practice* (this can be an end user practice but also a production system like a plant). The make plan makes it possible that (sooner or later) there is more than one instance (so if an inventor construes a technical object but there is no make plan and it cannot be reproduced, it is not an artefact instance).

3.2 Examples

Examples of artificial objects are not only hammers, cars and software, but also a footprint in the sand, or production waste. However, the latter two are not technical objects, as they are not intentionally made, and have no use plan. A pebble at the beach is a natural object but when cleaned and moved by a human to serve as doorstep, it becomes a technical object. When in some culture, pebbles are recognized as decent doorstops, then “pebble” is a variant of the doorstep artefact and a pebble individual adapted and used in this way can be said to conform to this artefact. A pebble has physical capacities like a certain weight and friction on which the attributed capacity of stopping a door builds.

Seedless bananas [33] are not technical objects, but there is a design for them, so “the seedless banana” is a non-technical artefact. A software program like Ms-Word is a technical artefact. An organizational system that includes humans (for instance, a defense system) is not a technical object. Whether it can be seen as conforming to a (non-technical) artefact “defense system type Y” requires in the first place that there is a design specification. So some systems are artefact instances, while others, for instance, a family, are not.

¹ “capability”, “potential” may be better terms but the literature uses the word “capacity”.

3.3 Conceptual model of the artefact and artefact network

We posit a *form* event behind any artefact instance, alternatively called making, producing, assembling, reproducing. The form event has a physical input and output, and an intentional input and output. The physical input consists of the components of the artefact, whereas the intentional input corresponds to the make plan (as part of the artefact design). The intentional output corresponds to the attributed capacity, the physical output is the thing made.

The main elements of the (technical) artefact are rendered in the conceptual model of Fig. 1 (grounding in UFO is not in this paper). Design specifies an artefact and consists of three parts (specifications). The design takes place in a conversation between user and designer – as such, it can be requested, proposed, accepted etc. It is also the designer and user who jointly and sometimes after a process of negotiation attribute capacities to the technical object. Note that user is a role. Not all individual users participate in the conversation. The conversation can be organized in different ways.

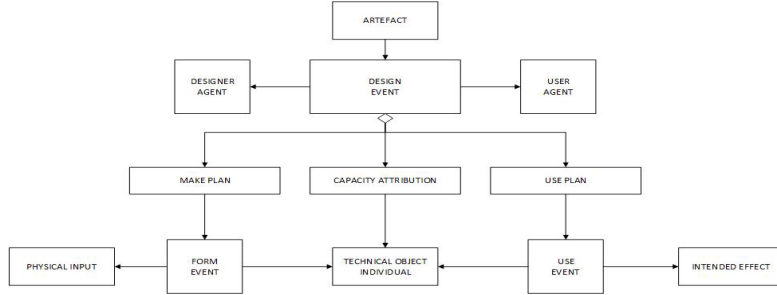


Fig. 1. Artefact ontology (technical artefact - conceptual overview).

The make plan *is not* a complete manufacturing plan. At the minimum, it specifies the critical components and their relationships (the artefact composition – system view). The make plan can take different forms, for instance, as a blueprint or a recipe. There is no absolute measure for the level of detail required. It should be sufficient for distinguishing the artefact from siblings (for instance, to distinguish the Diesel engine from the combustion engine).

The use plan consists of several parts. One important element is the intended effect. This effect can be specified more precisely if we consider a subset of technical artefacts that transform some input to some output – for instance, a coffee machine, a 3D printer, but also an application that transforms input data into output data. In that case, the intended effect can be described in the form of the required input and the intended output. We call these the use input and use output of the artefact, to avoid confusion with the make input and make output in the make plan.

In some sense, the artefact is a bridge between the make context and the use context. It is therefore not inappropriate to characterize an artefact as an interface [30].

An *artefact network* is a network of imbricating artefacts. The imbrication is a result of the make plan of one artefact being part of the use plan of another artefact. For instance, we can start with a mathematical Python library used in developing a Python

program. The library is part of the make plan of the program, while at the same time should comply with the use plan of the library. The software program can be part of an IS application that is used in a work system – the use plan of the program refers to the work practice, including resources and norms. The goal of the work system may be to build a robot. The use condition of the IS application may be that there is a running computer network that builds on Internet; etc. The implication for artefact design is that it always involves integrating a new artefact into the existing network, and this integration is an ongoing process as the network evolves continuously.

3.4 Text objects

A **text object** such as the blue book on my desk is a *technical object* where the attributed capacity (the written content) can be recaptured in the use (the reading). Text is to be understood in a wide sense. Any text object has a physical basis (e.g. paper, disk block), a content (the attributed capacity) and an intended use. The intended effect of using the text object (in the case of human reading) is typically a change in the mental state of the human agent corresponding to the attributed capacity (the text meaning). A software program at binary level is to be read by a processor, and the intended effect is a change in the internal memory of the computer.

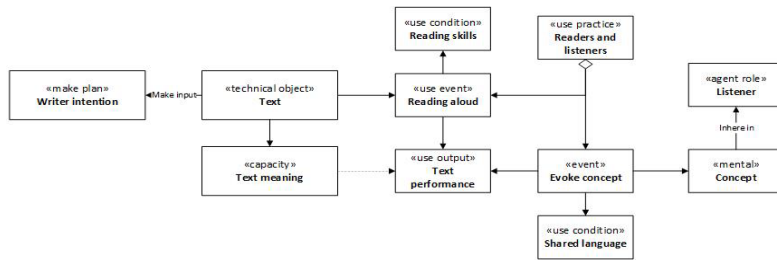


Fig. 2. Text for human reading (conceptual model)

We need to say something about the relationship between text and language. Although human language is not a technical object, text is. More in general, we can talk about writing technology that started with clay tablets and papyrus and nowadays is evolving into digital technology. Writing is a form event by which a technical (physical) object is created (the individual piece of paper, blank before the writing, and filled with signs after the writing). As for any technical object [32], the intended use must be grasped by the user. In line with the pragmatist slogan “meaning is use”, we consider the intended use (behavior) as the meaning of the text/sign. The intended behavior can be, for instance, to run away (in the case of a danger sign). The reading of the text for changing the mental state of the listeners is a special but of course very important use (Fig. 2). Its effectiveness depends (use condition) on the shared language and shared education in reading/writing. The meaning of the sign DOG in English is the evocation of the concept of dog shared by English-speaking people. This concept *is about* a certain kind of animals [8] but this is a property of natural language, not of the signs as such. When we go from the level of signs to the level of texts, then some texts are again

closely relate to behavior. For example, an IKEA manual that helps the customer to assemble a cabinet (the manual instantiates a make plan).

Examples of text objects are Mary's book, and Paul's computer chess game. Text objects may also be called symbolic objects. If the text object is not unique but conforming to an artefact, then the latter is a symbolic artefact. This artefact may survive when individual copies have been destroyed. The traditional distinction between data and information can be interpreted as a distinction between a technical object (the data item, as it may be written on multiple locations), and the artefact to which these data items conform and that specifies the attributed capacity and use plan of the data.

3.5 DSR artefact

A (domain) **Design Science Research (DSR) artefact** is an artefact with a well-described design specification (formal rigor) that is not only directly or indirectly used in a (domain - e.g. information system) practice for practical purposes but also in the research practice (the scientific community) for the purpose of building up knowledge. As science aims at abstract knowledge, not knowledge of the particular, the DSR artefact should generalize over multiple use contexts. So not every artefact is a DSR artefact. Not every software program is a IS design artefact. Not all design is design research (cf. [24]). DSR artefacts must have at least a certain level of *rigor* and *generalizability*, as well as practical and scientific *relevance*. Moreover, it must have instantiations so that evaluation knowledge can be acquired.

When rigor is required for the design, the specification must be explicit in the form of a text object. Ontologically, the relationship between artefact and design specification is one of inheritance (the former cannot exist independently from the latter). The specification is usually a composite object, consisting of several text objects. For instance, in the case of software design, one text object could be an object model.

4 An overview of IT/IS artefacts

Whereas some proposals have identified a shortlist of IT/IS artefacts and others have refrained from any characterization, our proposal is to identify a few core IT/IS artefacts and a generative mechanism by which secondary artefacts can be derived. It is possible, for instance, to focus on the design conversation and turn it into an (text) artefact; or to design a tool to support the conversation; or to make the use plan the focus of design research, in the form of a use method. March & Smith [23] mention constructs, models, methods and instantiations as IS design research artefacts, where "method" includes algorithms. Starting with this shortlist, in this section we identify IT/IS core artefacts.

4.1 Algorithms, programs and IS applications

According to [26] an algorithm, like a method, describes a sequence of activities that are executed by a computer. They define, it as 'an executable description of a system behavior'. Examples given are sorting algorithm, data mining algorithm, and protocol.

From the ontological perspective, the sorting algorithm, in one of its variants, e.g. quicksort (Fig.3), is an artefact that is instantiated in all its executable representations (text objects conforming to the algorithm artefact).

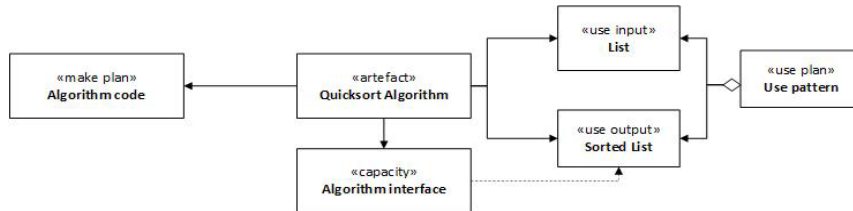


Fig. 3. The quicksort algorithm as an artefact.

In the pre-computer age, these executable representations had to be executed (on paper) by humans. The sorted list is created so it is a textual technical object. It has an attributed capacity (that the elements in the list are ordered) related to the use output.

In the computer age, the algorithm, in the form of software program, is executed by a computer processor. The use plan of this IT application requires there to be a processor and a database/storage. When the program is executed according to the use plan, a new database state is created. Just like using a manual means that the manual is executed step by step, so the use of the algorithm means that the make plan/algorithm code is executed.

We regard the algorithm as an artefact (*possibly* a DSR artefact) and a software program as a kind of algorithm. Apart from the difference in use plan (and related formal rigor required in the make plan), the structure of a software program is more complex than a sequence of computational steps. Nowadays, its composition is described in terms of objects or services. The way that the components are assembled by the make plan, has become a design object itself. Concrete approaches such as SOA are a DSR artefact.

A software program is used to change the internal state of the computer [35]. To make the bridge to the organizational context, we need I/O devices that are able to create text objects outside the machine. How the I/O device, software program and processor interact exactly is not in the scope of our analysis, but one way of viewing it is that the three are assembled into a new artefact, the IS application. This *IS application* is what is used by the human agent according to its use plan (the prescribed user interaction) to generate and transform data objects in the organization. This typically takes place as part of a work system or business process practice. When an IS application is assembled together with a machine or robot into a *cyberphysical system*, a new category of artefacts arises. However, most of these will not be within the IS domain, just like microelectronics falls outside the domain at the other side.

One ontological question is whether the code that is the make plan of the software artefact is an artefact itself. In principle, we prefer make plan and use plan of an artefact to be characterized as *artefact parts* – so that they are not evaluated in isolation. However, the form event governed by the make plan is an activity that is open to optimization and tool support, as any activity. Consider first a non-textual artefact like the Diesel

engine. The make plan of the metal object is a textual artefact, e.g. a blueprint. This text object can become the subject of design (of a text writer, addressing the construction workers), meeting user requirements of being easy to read, complete etc., and so becomes an artefact itself. For symbolic artefacts like a software program, the make plan (the blueprint) and made object (the code) *can* be one and the same. However, this is not necessary, and, as we know, it is possible to describe software programs on various levels of abstraction, e.g. in terms of design models, like an UML object model. “The UML object model” is a DSR artefact (see below for more discussion about models).

4.2 Supporting artefacts- formal languages, constructs and patterns

Supporting all text objects and in particular the software specification dimensions are formal language artefacts, like programming languages. A programming language provides the constructs (basic components) from which the source code can be assembled according to certain rules (use plan), that is, the syntax. [26] also mention the “pattern” as artefact, but the pattern has multiple levels. It often represents design knowledge about some artefact (and so not a design science artefact itself), but it can also be a semi-component design object and then it is indeed (on the type level) a genuine DSR artefact.

4.3 Models

In traditional system sciences, models are made of social, material, or socio-material situations, conceptualized as systems (cf. 2.1). Examples are models of the economic market, of a missile launch, or of (a part or aspect of) a business organization. The model can be analyzed mathematically or by simulation [30]. Based on the analysis, the manager of the organization can make decisions that influence the system, for instance, by introducing an IT system. In this case, we call the models analysis/design models. Whether the intervention will have the foreseen effect is not guaranteed: what is built is not the system as a whole, but certain elements of it. In the field of Information Systems, the manager can introduce new IT, and new methods (structuring the business processes), and in that way enhance the Information System in the broader sense. In terms of the ontology of artefacts, these systems are non-technical artefacts, if there is a design specification, with a make plan and use plan, and these plans may take the form of a “design model” or “use model”. In the near future, we expect these kind of analysis/design models to play a role in autonomous intelligent systems (robots). Not only for building these systems, but also for these systems themselves to reason about their environment.

In many situations, such as a meteorological system, there is no design specification, but we can have a model for analysis and prediction purposes.

Let us consider the example of an Entity Relationship diagram. It has a dual nature. On the one hand, it can be used as an analysis model of a non-technical system, for instance, a household. In most cases, it is an analysis/design model of a non-technical artefact, for instance, a university domain consisting of students, enrollments etc., that wants to build an IS. On the other hand, it is used to build a database system, a technical

artefact. This design takes place in the conversation between system designer and user (that is why the graphical representation is useful) and the ER model is the make plan of the IT application. So: database systems are artefacts. “The ER model” as introduced by Chen is an artefact. The use plan of the ER model imbricates with the make plan of the database, while the use plan of the IT application (with database) is imbricates with the make plan of the IS (the analysis/design model of the IS).

We can go one step further and consider the purpose of the model. According to [4:693], ‘the models developed in the engineering sciences should be distinguished from the models produced in engineering [in technological practice]. Whereas the latter usually represent the design of a device or its mechanical workings, models in the engineering sciences aim for scientific understanding of the behavior of different devices or the properties of diverse materials’ (cf. [10,25]).

The intended use of the model for scientific understanding resonates with the design *science* perspective, although the distinction between practical and scientific can be diffuse. It is fair to say that the scientific rigor of the current DSR models is often lower than in the exact sciences with their long mathematical tradition. However, this does not invalidate the DSR ambition. In Transformational Design Research, the conversation for design between designer and user is distinguished from the conversation between the designer-researcher and the scientific community. Once a model is part of the latter conversation, it may be called a techno-scientific object. At the type level we talk about DSR artefacts: the model in the form of a design text conforms to a DSR artefact at type level, with make plan, use plan and attributed capacities of the model.

Summarizing, we have to distinguish different kinds of models. They are all technical objects, but they have different relationships to artefacts.

- *Analysis models of (socio-technical or other) system.* Some of these are IS artefacts. Examples are domain models that aim to analyze the behavior, for instance, system-dynamics simulations. The model instantiates “the system-dynamics” artefact.
- *Analysis/design models of non-technical artefacts.* Examples are BPMN business process models used in IS design (part of use plan). Another example can be an environment model that intelligent agents use for internal reasoning.
- *Design models of technical artefacts.* For instance, ER model. The model is both a make-plan for the technical artefact database, and instantiates “the ER model” artefact.
- *Scientific models as DSR artefacts.* Example is the use of Petri Net models for analyzing processes within technical or non-technical systems; or Codd’s relational database model grounded in set theory.

4.4 Summary

Our claim is that *software program* and *IS application* are the core domain artefact types in Computer Science and Information Systems. The specification of these artefacts can include *models* and *formal languages/constructs*. Instantiated systems [23] are *not* artefacts in our ontology, but evaluation studies of these technical objects (or

systems) contribute to the design knowledge accumulated in the artefact to which the individual conforms. From a Design Science perspective, *techno-scientific models* of the domain artefacts and the related systems are artefacts on a second level that typically focus on some aspect of the domain artefact. Our ontological analysis confirms the centrality of most artefact types mentioned by March & Smith, but with several nuances. We have made a key distinction between software program and IS application. We also claim that always some element of a design can be isolated and give rise to new artefact types. Furthermore, we have argued for a holistic view, that is, to see artefacts always in context of the artefact network.

4.5 Practical implications

Based on our ontological analysis, we can give some practical implications for the designer and design researcher in the form of a preliminary set of guidelines.

- Keep the difference between technical objects and non-technical objects.
- Focus not on building an artefact, but on designing an artefact (where designing includes the evolution of the design). Evaluate the artefact on the basis of its built instances in their use context to accumulate design knowledge.
- Include in the artefact design some make plan that specifies the composition
- Include in the artefact design one or more use plans that specify use conditions, intended effect and relate to a use practice.
- Perform the design always in conversation with actual or potential users. Identify attributed capacities together and analyze the relationship between inherent and attributed capacities.
- Integrate the designed artefact in the artefact network (aggregation dimension) and in the artefact classification (identify type and distinguish it from siblings)
- (for the design researcher) Aim at developing techno-scientific models of domain artefacts

5 Conclusion

Artefacts are said to be the object of Design Science Research, but their ontological status is often taken for granted. This has led to discussions and confusion about the artefact status of information systems. In this paper, we have proposed some important distinctions, in particular the distinction between a technical object (individual) and the artefact (universal) and the distinction between technical artefacts and non-technical artefacts. Whereas the literature often isolates either the designer intention or the artefact use, our ontology keeps the two in balance. We have founded IS artefacts on writing technology.

A new element in our ontology is to let the design specification inhere in a conversation between designers and users, a conversation that has several phases, in fact, an ongoing conversation as long as the artefact persists. Another aspect that we emphasize is the imbrication structure of the artefact network. A general conclusion of our analysis

is that design research should broaden its narrow scope from building/evaluating an isolated technical object in some delineated time span, at some place towards a holistic view on technology development.

We are currently working on a formal presentation of our artefact ontology grounded in UFO.

References

1. Aken, J. van: Management Research as Design Science: Articulating the Research Products of Mode 2 Knowledge Production in Management. *British J. of Mgt* 16:19-36 (2005).
2. Alter, S.: The concept of 'IT artifact' has outlived its usefulness and should be retired now. *Information Systems Journal* 25(1) (2015).
3. Baker, L.R.: The ontology of artefacts. *Philosophical explorations*, 7(2):99-111(2004).
4. Boon, M., Knuuttila, T.: Models as epistemic tools in engineering sciences. In A. Meijers (Ed.), *Philosophy of technology and engineering sciences*, pp. 693–726 (2009).
5. Borgo, S., Vieu, L: Artefacts in formal ontology. *Philosophy of technology and engineering sciences*, 273-307 (2009).
6. Carvalho, V, Almeida, J., Fonsceca, C., Guizzardi, G.: Extending the Foundations of Ontology-Based Conceptual Modeling with a Multi-level Theory. *ER 2015. Springer, LNCS 9381*, pp119-133. (2015).
7. Ceusters, W.: An information artifact ontology perspective on data collections and associated representational artifacts. *Stud Health Technol Inform* (180) 68-72(2012).
8. Ceusters, W., Smith, B.: Aboutness: Towards Foundations for the Information Artifact Ontology. *Proc. 6th Int. Conf on Biomedical Ontology. CEUR vol. 1515*. pp. 1-5. (2015).
9. Drechsler, A., Hevner, A.: Utilizing, producing, and contributing design knowledge in DSR projects. *Proc. DESRIST 2018*, pp. 82-97. Springer (2018).
10. France, B., Compton, V. J., Gilbert, J. K. Understanding modelling in technology and science: The potential of stories from the field. *Int. Journal of Technology and Design Education*, (21) 381–394 (2010).
11. Goldkuhl, G., Lind, M.: A multi-grounded design research process. *Proc. DESRIST 2010*, pp. 45-60. Springer (2010).
12. Grabowski, F., & Strzalka, D.: Simple, complicated and complex systems — the brief introduction. *2008 Conference on Human System Interactions*, 570-573 (2008).
13. Jones, D., & Gregor, S.: The Anatomy of a Design Theory. *J. AIS*, 8, 19. (2007).
14. Gregor, S., Hevner, A.: Positioning and presenting design science research for maximum impact. *MIS Q.* 37(2): 337–356 (2013).
15. Guizzardi, G., de Almeida Falbo, R., Guizzardi, R.S. Grounding Software Domain Ontologies in the Unified Foundational Ontology (UFO): The case of the ODE Software Process Ontology. In: *Proc. CIBSE*, pp. 127–140. (2008).
16. Hevner, A., March, S., Park, J. and Ram, S.: Design Science in Information Systems Research'. *MIS Q.* 28(1): 75-105 (2004).
17. Iivari, J.: A paradigmatic analysis of information systems as a design science. *Scand. J. Information Systems* 19:39-64 (2007).
18. Johannesson, P., Perjons, E.: *An Introduction to Design Science*. Springer (2014).
19. Kimbell, L.: Rethinking design thinking: Part I. *Design and Culture*, 3(3), 285-306 (2011).
20. Kroes, P., Meijers, A.: The dual nature of technical artifacts, *Studies in History and Philosophy of Science*, 37(1): 1–4 (2006).

21. Lawson, C.: An Ontology of Technology: Artefacts, Relations and Functions. *Techné* 12(1):48-64 (2008).
22. Lee, A., Thomas, M., Baskerville, R.: Going back to the basics in design science: from the information technology artefact to the information system artefact. *Information Systems* 25:5-21 (2014).
23. March, S., Smith G.: Design and natural science research on information technology. *Decision Support Systems* 15:251-266 (1995).
24. McPhee, K.: Design Theory and Software Design. Technical report TR 96-26, Univ of Alberta. (1997).
25. Nia, M., de Vries, M.: Models as artefacts of a dual nature: a philosophical contribution to teaching about models designed and used in engineering practice. *Int. Journal of Technology and Design Education*, 27(4), 627-653 (2017).
26. Offermann, P. Blom, S., Schönherr, M., Bub, U.: Artefact types in information system design science – a literature review. In: Winter R et al (eds) *Global perspectives on design science research*, Springer, pp.77-92. (2010).
27. Pandza, K., Thorpe, R.: Management as Design, but What Kind of Design? An Appraisal of the Design Science Analogy for Management. *British J. of Mgt*, v. 21, p. 171-186, (2010).
28. Purao, S.: Design Research in the Technology of Information Systems: Truth or Dare. GSU Dept of CIS Working Paper, Atlanta: Georgia State University. (2002).
29. Silver, M., Markus, L.: Conceptualizing the SocioTechnical (ST) Artifact. *Systems, Signs & Actions*, 7(1), 82-89. (2013).
30. Simon, H.: *The Sciences of the Artificial* (3rd edition), MIT Press. (1996).
31. Simondon, G.: *On the Mode of Existence of Technical Objects* Minneapolis: Univocal Publishing. (1958/2016).
32. Sinha, Ch.: Language and other artifacts. *Frontiers in Psychology* (6):1601. (2015).
33. Sperber, D.: Seedless Grapes: Nature and Culture. In: Margolis, E., and Laurence, S. (eds.), *Creations of the Mind: Theories of Artifacts and Their Representation*, Oxford: Oxford University Press, pp. 124–137 (2007).
34. Vermaas, P., Kroes, P., Poel, van de, I., Franssen, M., Houkes, W.: *A philosophy of technology: from technical artefacts to sociotechnical systems*. Synthesis lectures on engineers, technology and society 6(1), San Rafael: Morgan & Claypool Publishers. (2011).
35. Wang, X., Guarino, N., Guizzardi, G., Mylopoulos, J.: Towards an Ontology of Software: a Requirements Engineering Perspective. *Proc. FOIS 2014*, pp. 317–329. IOS Press (2014).
36. Weigand, H.: Value Expression in Design Science Research. *IEEE RCIS 2019*: 1-11 (2019).
37. Wieringa, R.: *Design Science Methodology*. Springer, (2013).
38. Winter, R.: Design Science Research in Europe *European J. of Information Systems* 17:470-474 (2008).